

INFORME FINAL — ERSO 2026-002

- [INFORME FINAL — ERSO 2026-002](#)
 - [Suite criptográfica y formatos de credencial: SD-JWT VC y mdoc](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 002, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los seis pasos](#)
 - [10. Los cuatro criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO](#)
 - [13. Mapa de requisitos a implementación](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación](#)
 - [16. Laboratorio A — Paso 1: el perfil por formato](#)
 - [17. CRITERIO 1 — Emisión y recepción en dc+sd-jwt y mso_mdoc](#)
 - [18. CRITERIO 2 — Los algoritmos y formatos declarados coinciden con los efectivamente usados](#)
 - [19. CRITERIO 3 — No existe ningún protocolo propio paralelo](#)
 - [20. CRITERIO 4 — La firma del emisor está separada de la prueba del titular](#)
 - [21. Pruebas automáticas](#)
 - [22. Recorrido de extremo a extremo \(extracto del registro\)](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [23. Preguntas sobre los conceptos](#)
 - [24. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [25. Redacción modelo de las cuatro respuestas](#)
 - [26. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [27. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [28. Si algo no sale como en el informe](#)
 - [29. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-002

Suite criptográfica y formatos de credencial: SD-JWT VC y mdoc

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-002 — Suite criptográfica y formatos de credencial: SD-JWT VC y mdoc
Desarrollador asignado (según la ERSo)	Luis González
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Geovani Rincón
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: miércoles 23-sep-2026, 7:30 a. m. · Prueba: viernes 25-sep-2026, 7:30 a. m.
Dominio del laboratorio	civica-desarrollo.avance.org.co (emisor y verificador: puerto 8444)
Ubicación del proyecto	/home/geovani/Descargas/generic/bitacora/new_proyect
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	scripts/lab-002-completo.sh · informes/ERSo-2026-002.md (versión corta) · docs/PERFIL-CREDENCIALES.md · docs/ANALISIS-BASES-EUDI.md

Nota de fechas. El desarrollo (23-sep) y la prueba (25-sep) son anteriores a la fecha de este informe (1-oct-2026). El documento sirve para repetir la prueba o dejar constancia; las firmas de la tabla de actividades son de las personas responsables.

Lo más importante que debes saber antes de empezar. Emisor, verificador y cartera fueron escritos **por el mismo equipo** leyendo la misma especificación. Que se entiendan entre sí **no demuestra** interoperabilidad con implementaciones de terceros. Este informe lo dice con todas sus letras en cada lugar donde corresponde y lo resume en §27. Tampoco se reutilizó ninguna biblioteca de credenciales: la implementación es propia sobre el JDK y una biblioteca CBOR.

Cómo leer este informe

Símbolo	Significa
💡	Idea clave: lo más importante de la sección, en una frase
🔍	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
✅	Qué significa: cómo interpretar lo que el sistema respondió

Símbolo	Significa
⚠	Cuidado: una trampa frecuente o un límite que conviene conocer
🔧	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII
Quien necesita entender el problema	Parte I y Parte II
Quien debe reproducir y comprobar	Partes III y IV
Quien prepara la prueba funcional	Partes IV, VI y la lista de comprobación (§26)
Quien prepara una exposición	Partes V y VI

Cómo están presentados los comandos. Los bloques `console` son una **sesión de terminal real**: las líneas con `$` son lo que se escribe; las demás, lo que el sistema respondió. Los valores que cambian en cada ejecución (identificadores, claves, tokens, códigos, `iat`) serán distintos al repetir; lo que debe coincidir es el **patrón**. Los códigos y tokens que aparecen son de laboratorio y de corta vida.

Un recurso de este informe: «el recorrido». Se añadió al simulador del titular (`holder-sim`) el comando `scenario tour-credentials`, que ejecuta el flujo completo de emisión y presentación **imprimiendo cada artefacto**: la credencial decodificada por partes, las divulgaciones, el KB-JWT, el registro de ejecución. No cambia nada del producto; solo muestra. Se ejecuta una vez y su salida se reparte en las secciones.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Los términos, en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó, cómo se monta en Docker y cómo funciona por dentro
IV · Laboratorios	14 a 22	La prueba material de cada criterio, con cada comando y su respuesta
V · Preguntas y respuestas	23 y 24	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	25 y 26	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	27 a 29	Lo que NO se demuestra, qué hacer si algo falla y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Una credencial verificable es, en esencia, una afirmación firmada («Ana cursó Ingeniería de Sistemas») que el titular guarda en su cartera y presenta a quien se la pida. Para que funcione entre organizaciones distintas hay que resolver cuatro cosas a la vez:

1. **Formato:** la credencial debe estar escrita de una manera que otros entiendan. Hoy hay dos formatos dominantes: **SD-JWT VC** (basado en texto/JSON) y **mdoc** (basado en binario/CBOR, el de la licencia de conducir móvil).
2. **Protocolo:** cómo se entrega (emisión) y cómo se pide y responde (presentación). Existen estándares: **OpenID4VCI** y **OpenID4VP**. Inventar los propios rompería la interoperabilidad.
3. **Perfil:** dos sistemas interoperan cuando coinciden en el **detalle** (qué algoritmo, qué cabecera, qué forma de clave), no solo en que ambos «hablen de credenciales».
4. **Separación de firmas:** la firma del **emisor** (valida la credencial) y la prueba del **titular** (valida quién la presenta) deben ser distintas e independientes.

La solución. Un **servicio de credenciales** con un emisor OpenID4VCI y un verificador OpenID4VP, un **perfil declarado en un solo lugar** que un **registro de ejecución** comprueba contra lo realmente usado, un **inventario de endpoints** que prueba que no hay protocolos paralelos, y una **matriz de claves por rol**.

El resultado.

#	Criterio	Resultado	Dónde
1	Emisión y recepción en <code>dc+sd-jwt</code> y <code>mso_mdoc</code> conforme al perfil	✓ entre nuestros componentes · ⚠ no contra terceros	§17
2	Algoritmos y formatos declarados = usados	✓	§18
3	Ningún protocolo propio paralelo	✓	§19
4	Firma del emisor separada de la prueba del titular	✓	§20

Cifras: 28 pruebas automáticas propias (12 + 16, todas en verde); una credencial decodificada por partes con sus 4 divulgaciones y su digest verificado uno a uno; 9 rutas típicas de «protocolo propio» sondeadas, todas `404`.

Lo que debes saber de antemano: (1) no se probó contra emisores de terceros (EUDI); (2) la clave del emisor se descubre por **DID**, no por certificado `x5c`, así que un verificador EUDI estándar no validaría estas credenciales sin adaptación; (3) no se implementó la presentación de mdoc, ni cifrado JWE, ni DPoP, ni revocación (§27).

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

1. La historia en cinco minutos

Imagina que **una universidad** (el *emisor*) entrega un **diploma** a Ana (la *titular*), que lo guarda en su **cartera**. Más tarde, una **empresa** (el *verificador*) le pide a Ana que demuestre que estudió Ingeniería de Sistemas.

- El diploma lleva el **sello de la universidad** (la firma del emisor): cualquiera puede comprobar que lo emitió la universidad.
- El diploma tiene varios datos (nombre, programa, promedio) pero **Ana decide cuáles enseña**: puede mostrar solo el programa y mantener ocultos su nombre y su promedio (*divulgación selectiva*). Los datos ocultos siguen «cubiertos» por el sello.
- Al presentarlo, Ana **firma además una nota** —«se lo muestro a esta empresa, ahora, con este código»— con **su propia llave**, la que quedó ligada al diploma. Así la empresa sabe que quien lo presenta es **la dueña**, y que no es una copia robada reenviada.
- La universidad y la empresa no necesitan conocerse: usan **formatos y protocolos estándar**.

💡 Hay **dos firmas distintas**: la del **emisor** (prueba que el diploma es auténtico) y la de la **titular** (prueba que lo presenta su dueña). Si se confundieran, la universidad podría fabricar presentaciones, o cualquiera que robe un diploma podría usarlo.

2. Diccionario de términos

2.1 Los roles

Término	En palabras sencillas
Emisor (<i>issuer</i>)	Quien firma y entrega la credencial (aquí, Avance).
Titular (<i>holder</i>)	Quien la recibe y la guarda en su cartera.
Verificador (<i>verifier</i>)	Quien pide una presentación y la comprueba.
Credencial verificable	Una afirmación firmada por el emisor sobre el titular.
Presentación	Lo que el titular enseña al verificador (puede ser solo una parte).
Cartera (<i>wallet</i>)	La aplicación del titular que guarda y presenta credenciales.

2.2 Formatos

Término	En palabras sencillas
SD-JWT	<i>Selective Disclosure JWT</i> : un JWT firmado cuyos datos personales están ocultos tras resúmenes, y que se acompaña de las «divulgaciones» que el titular decida enseñar.

Término	En palabras sencillas
SD-JWT VC	La versión del SD-JWT pensada para credenciales verificables. Su identificador de formato es <code>dc+sd-jwt</code> .
Divulgación (disclosure)	Un dato oculto: <code>[sal, nombre, valor]</code> codificado. El titular decide si lo incluye.
Digest (resumen)	La huella (sha-256) de una divulgación, colocada dentro del JWT bajo <code>_sd</code> . Prueba que el dato ocultado estaba cubierto por la firma del emisor.
Sal (salt)	Un aleatorio dentro de cada divulgación para que no se pueda adivinar su contenido por su resumen.
<code>cnf</code>	«Confirmación»: la clave pública del titular a la que queda ligada la credencial.
<code>vct</code>	Tipo de credencial (<i>verifiable credential type</i>), p. ej. <code>urn:avance:credential:academic:1</code> .
KB-JWT	<i>Key Binding JWT</i> : la prueba del titular al presentar: lleva <code>aud</code> (para quién), <code>nonce</code> (código de la solicitud), <code>iat</code> (cuándo) y <code>sd_hash</code> (resumen de lo presentado), firmada con la clave de <code>cnf</code> .
mdoc / mso_mdoc	Formato de la norma ISO/IEC 18013-5 (licencia de conducir móvil): estructura binaria CBOR .
CBOR	Un formato binario compacto, primo de JSON.
MSO	<i>Mobile Security Object</i> : el objeto dentro del mdoc con los resúmenes de todos los datos, firmado por el emisor.
COSE_Sign1	La forma de firma del mundo CBOR (primo de JWS).
<code>docType</code>	El tipo de documento mdoc, p. ej. <code>org.avance.academic.1</code> .
<code>deviceKey</code>	La clave del titular dentro del mdoc (equivale a <code>cnf</code>).

2.3 Protocolos

Término	En palabras sencillas
OpenID4VCI 1.0	El estándar para emitir credenciales: oferta → token → nonce → credencial.
OpenID4VP 1.0	El estándar para presentarlas : solicitud → respuesta del titular.
Oferta de credencial	El mensaje con el que el emisor invita al titular a recoger una credencial.
Código pre-autorizado	Un código de un solo uso que va dentro de la oferta y permite pedir el token sin iniciar sesión.
<code>tx_code</code>	Un código corto adicional (p. ej. 6 dígitos) que el titular recibe por otro canal; protege contra quien intercepte la oferta.
Token de acceso	El pase que permite pedir la credencial.
Nonce de emisión (c_nonce)	Valor de un solo uso que el titular debe firmar en su prueba de posesión.
Prueba de posesión (de emisión)	Un JWT <code>openid4vci-proof+jwt</code> firmado con la clave que quedará en <code>cnf</code> .
DCQL	Lenguaje de consulta de OpenID4VP: «quiero una credencial de este tipo y este dato».

Término	En palabras sencillas
direct_post	Modo de respuesta: la cartera envía la respuesta por POST directo al verificador.
Metadatos del emisor	Documento público (/ .well-known/...) que anuncia qué formatos, algoritmos y endpoints ofrece.

2.4 Interoperabilidad y perfil

Término	En palabras sencillas
Interoperar	Que dos sistemas independientes se entiendan sin ajustes.
Perfil	La elección concreta de opciones dentro de un estándar: formato, tipo de cabecera (typ), algoritmos, forma de las claves.
Algoritmo ES256	ECDSA con curva P-256 y SHA-256. En COSE se identifica con el número -7 .
Registro de ejecución	Una lista de lo que realmente se usó (formato, algoritmo, resumen) al emitir y verificar, comparada con lo declarado.
Inventario de endpoints	La lista de rutas que expone el servicio, cada una con su norma.
Protocolo paralelo	Un camino propio, no estándar, para emitir o presentar (p. ej. /issue , /present). Está prohibido.
Matriz de claves por rol	Tabla que dice, para cada rol, qué clave usa, dónde vive su parte privada y quién la verifica.
EUDI / HAIP	El ecosistema europeo de carteras y su perfil de alta garantía; son la referencia con la que se compara.
x5c	Cadena de certificados X.509 con la que el ecosistema EUDI identifica al emisor. Este proyecto usa el DID en su lugar.

3. Las siete ideas madre

1. **Interoperar es coincidir en el detalle**, no solo en el tema.
2. **Un solo perfil, escrito en un solo lugar** y comprobado contra lo realmente usado.
3. **Solo protocolos estándar**. Nada de rutas propias.
4. **Dos firmas, dos claves, dos verificaciones**.
5. **El titular decide qué enseñar**; lo oculto sigue cubierto por la firma.
6. **La cartera no confía en lo recibido**: verifica la firma del emisor (resolviendo su DID) antes de guardar.
7. **Declarar es una promesa; el registro de ejecución es la prueba**.

4. El método de trabajo: cómo se piensa un criterio

1. **Descomponer** el criterio en el verbo y la evidencia.
2. **Hacerlo comprobable**: ¿qué comando lo demuestra?
3. **Buscar el caso negativo**.
4. **Predecir**.
5. **Ejecutar y observar**.

6. **Correlacionar** con la regla del código.

7. **Redactar** la respuesta con la evidencia **y con el límite** si lo hay.

PARTE II — QUÉ PIDE LA ERSO 002, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Suite criptográfica y formatos de credencial: SD-JWT VC y mdoc
Desarrollador	Luis González
Fecha de la ERSO	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Miércoles, 23 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Viernes, 25 de septiembre de 2026, 7:30 a. m.
Responsables	Análisis y diseño: Karen Flórez Madiedo · Asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Luis González · Plantilla de pruebas y pruebas funcionales: Geovani Rincón

⚠ Las firmas y fechas de la tabla de actividades **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

Capacidades → Condiciones → Descripción → Qué debe hacer (6 pasos) → Criterios (4) → Actividades

6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar la **emisión remota** de credenciales verificables en formato **dc+sd-jwt** y la compatibilidad con **mso_mdoc**, fijando los **perfiles por formato**, los **algoritmos** y la **separación entre la firma del emisor y la prueba del titular, sin crear protocolos propios paralelos**."

#	Fragmento	En lenguaje sencillo	Cómo se materializó
1	" emisión remota "	Entregar la credencial por la red, sin presencia física	OpenID4VCI 1.0
2	" dc+sd-jwt "	Emitir en el formato SD-JWT VC	SdJwtVcIssuer

#	Fragmento	En lenguaje sencillo	Cómo se materializó
3	"compatibilidad con <code>mso_mdoc</code> "	Poder recibir y guardar credenciales mdoc	<code>MdocVerifier</code>
4	"perfiles por formato y algoritmos"	Fijar cada detalle y declararlo	<code>CredentialProfiles</code> + documento de perfil
5	"separación firma del emisor / prueba del titular"	Dos firmas, dos claves	<code>KeyMatrix</code>
6	"sin protocolos propios paralelos"	Solo estándares	Inventario de endpoints

6.2 Las cinco capacidades

#	Capacidad (texto del PDF)	Qué se busca	Cómo se resolvió	Paso
1	<i>Emitir credenciales verificables en <code>dc+sd-jwt</code></i>	Que la cartera reciba una SD-JWT VC válida	Emisor OpenID4VCI	2
2	<i>Recibir y almacenar credenciales en <code>mso_mdoc</code></i>	Que la cartera acepte y guarde un mdoc verificado	<code>MdocVerifier</code> + almacén de la cartera	3
3	<i>Fijar y declarar los perfiles por formato y los algoritmos admitidos</i>	Un perfil escrito que coincida con lo usado	<code>Profiles.kt</code> , <code>PERFIL-CREDENCIALES.md</code> , registro de ejecución	1
4	<i>Separar la firma del emisor de la prueba de posesión del titular</i>	Dos claves distintas	<code>KeyMatrix</code>	4
5	<i>Corregir las diferencias de perfil detectadas en las bibliotecas reutilizadas</i>	Alinear con el estándar	No se reutilizó ninguna: perfil propio, diferencias documentadas	5

7. Condiciones del proceso

Condición 1 — "Se adoptan las versiones fijadas de OpenID4VCI 1.0 y OpenID4VP 1.0."

- **Qué significa.** Se sigue una versión concreta de cada estándar, no «lo que haya».
- **Qué problema evita.** Que dos implementaciones usen versiones distintas y no se entiendan.
- **Cómo se cumple.** Los metadatos del emisor y los endpoints siguen las secciones citadas de las versiones 1.0; las referencias están en `docs/VERSIONES-NORMATIVAS.md`.
- ⚠️ Varios detalles están escritos de memoria del texto final de la especificación y **deben confirmarse** contra el texto exacto (§27).

Condición 2 — "No se crean protocolos propios paralelos a OpenID4VCI ni a OpenID4VP."

- **Qué significa.** Todo el intercambio de credenciales usa los endpoints estándar.
- **Qué problema evita.** Sistemas que solo se entienden con su propio fabricante.
- **Cómo se cumple.** Inventario de endpoints con su norma; sondeo de rutas típicas (§19).

Condición 3 — "Los perfiles y algoritmos declarados deben coincidir con los efectivamente usados."

- **Qué significa.** No se puede declarar ES256 y firmar con otro algoritmo.
- **Qué problema evita.** Una declaración «de adorno» que no refleja lo que ocurre.
- **Cómo se cumple.** Un **registro de ejecución** que lee los artefactos reales (§18).

8. Descripción del proceso

"El sistema debe emitir y aceptar credenciales verificables en los formatos que ya soporta el ecosistema (SD-JWT y mdoc) y transportarlas con los protocolos estándar de emisión y presentación."

Dos formatos, dos protocolos.

"El propósito es lograr interoperabilidad real: dos implementaciones interoperan cuando coinciden en el perfil concreto de formato y algoritmos, no solo porque ambas «hablen» de credenciales."

La frase clave: interoperar es coincidir en el **detalle**. De ahí que el perfil se fije, se documente y se compruebe.

"Literatura y temas a consultar"

Lectura	Qué te aporta
SD-JWT y SD-JWT VC	El formato de la credencial con divulgación selectiva
ISO/IEC 18013-5 (mDL) y mso_mdoc	El formato binario del mdoc
OpenID4VCI y OpenID4VP	Los protocolos de emisión y presentación
W3C VC Data Model 2.0	El modelo general de credencial
Perfiles de interoperabilidad (DIF) y vectores de prueba	Cómo se comprueba que dos sistemas interoperan

9. Qué debe hacer: los seis pasos

1. Fijar y documentar el perfil por formato (dc+sd-jwt y mso_mdoc) y la lista de algoritmos admitidos.
2. Implementar la emisión de credenciales en dc+sd-jwt conforme al perfil.
3. Implementar la recepción y el almacenamiento de credenciales en mso_mdoc cuando el emisor las ofrezca.
4. Separar en la implementación la firma del emisor de la prueba de posesión del titular.
5. Corregir las diferencias de perfil detectadas en las bibliotecas reutilizadas.

6. Verificar que no se introduce ningún protocolo propio paralelo a OpenID4VCI u OpenID4VP.

Paso	Lo que pide	Criterio que lo comprueba	Laboratorio
1	Fijar y documentar el perfil	Criterio 2	§16 y §18
2	Emisión en <code>dc+sd-jwt</code>	Criterio 1	§17
3	Recepción y almacenamiento de <code>mso_mdoc</code>	Criterio 1	§17
4	Separar firma del emisor y prueba del titular	Criterio 4	§20
5	Corregir diferencias de perfil	(documentado, §27)	§27
6	Verificar que no hay protocolo propio	Criterio 3	§19

10. Los cuatro criterios de aceptación

1. La emisión y la recepción funcionan en `dc+sd-jwt` y `mso_mdoc` conforme al perfil adoptado; evidencia: pruebas cruzadas con OpenID4VCI 1.0 y OpenID4VP 1.0. **2.** Los algoritmos y formatos declarados coinciden con los efectivamente usados; evidencia: documento de perfil por formato y registro de ejecución. **3.** No existe ningún protocolo propio paralelo a OpenID4VCI ni OpenID4VP; evidencia: inventario de endpoints y revisión de código de integración. **4.** La firma del emisor está separada de la prueba del titular; evidencia: matriz de claves por rol y pruebas de verificación independientes.

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo
1	<i>Funcionan</i> · pruebas cruzadas	Emisión y recepción de ambos formatos; presentación con divulgación selectiva	<code>tx_code</code> erróneo; credencial alterada; repetición	OpenID4VCI/VP + verificadores
2	<i>Coinciden</i> · perfil y registro	Perfil escrito + registro de lo usado sin diferencias	Un ES384 o un formato no declarado se detecta	ExecutionLog
3	<i>No existe</i> · inventario y revisión	Lista de endpoints con su norma; rutas típicas = 404	Intentar <code>/issue</code> , <code>/present</code> ...	Inventario + nginx
4	<i>Está separada</i> · matriz y verificaciones	Dos claves distintas; cada firma solo valida con su clave	Verificar con la clave equivocada	KeyMatrix

⚠ Observación sobre el criterio 1. Dice «pruebas cruzadas con OpenID4VCI 1.0 y OpenID4VP 1.0». Aquí las pruebas se cruzan entre **nuestro** emisor/verificador y **nuestra** cartera; **no** contra implementaciones de terceros. Es el siguiente paso para que el criterio sea concluyente (§27).

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO

11. Visión general

11.1 Qué se construyó

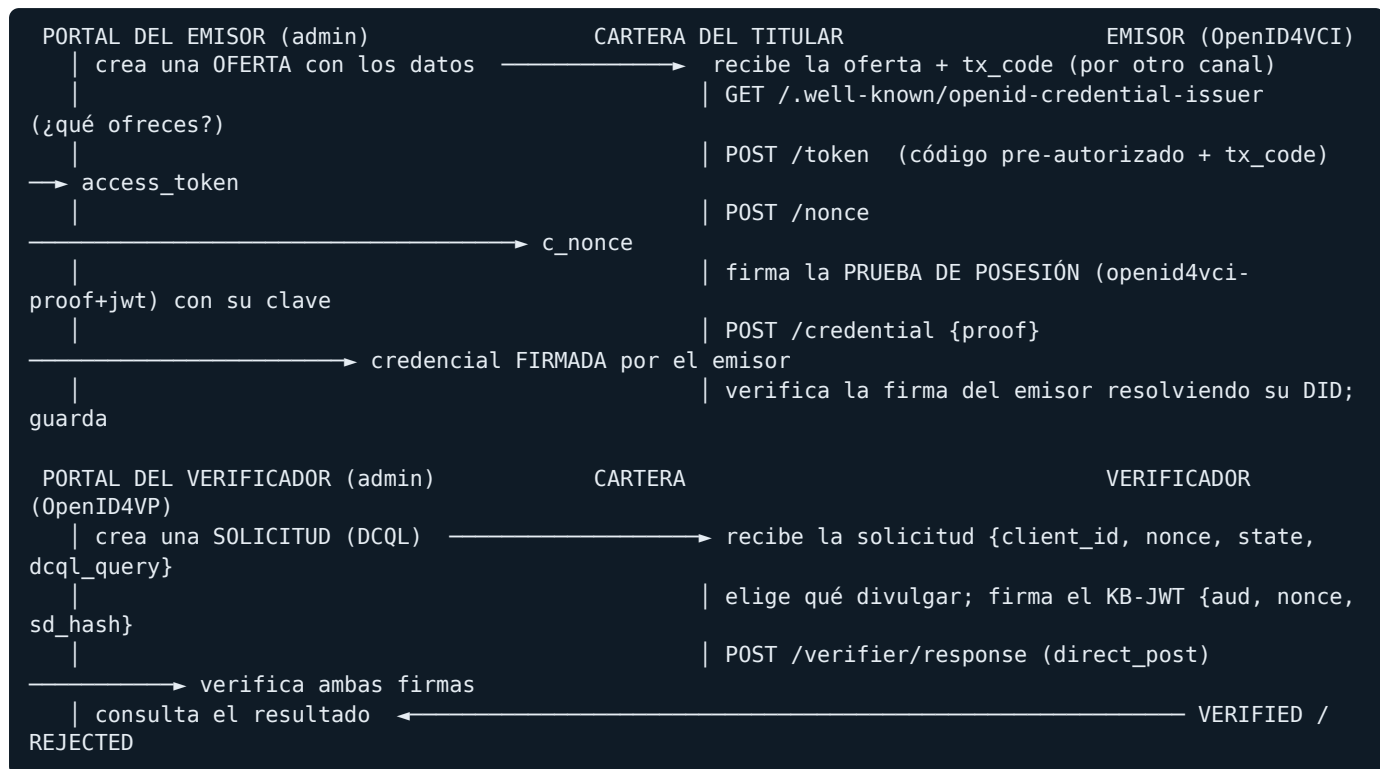
Pieza	Dónde corre	Qué hace
Servicio de credenciales (credential-service)	Contenedor credential	Emisor OpenID4VCI 1.0 + verificador OpenID4VP 1.0 + portales de administración
credentials-core	Biblioteca	Formatos: SdJwtVcIssuer/Holder/Verifier , MdocVerifier , Jwk , KeyMatrix , ExecutionLog , CredentialProfiles
Cartera del titular (CredentialWallet)	holder-sim (contenedor holder , perfil tools)	Canjea ofertas, verifica la firma del emisor , guarda y presenta
did-resolver	Biblioteca (ERSo 007)	Resuelve el DID del emisor para obtener su clave pública
Clave del emisor	evidencias/work/avance.json (laboratorio)	Firma las credenciales; su pública está en el DID de Avance (ERSo 006)

11.2 Las piezas en Docker

Contenedor	Para qué sirve en esta ERSO
nginx	Puerta 8444 : solo los endpoints estándar; los portales de administración van por 9443 con certificado cliente
credential	El emisor/verificador (puerto interno 8100; no se publica)
vdr	Sirve el DID del emisor; sin él la cartera no podría verificar la firma
holder (perfil tools)	La cartera simulada

```
$ docker compose ps --format 'table {{.Service}}\t{{.Status}}\t{{.Ports}}'
SERVICE      STATUS          PORTS
credential     Up 3 minutes
nginx          Up 3 minutes    80/tcp, 0.0.0.0:8444->8444/tcp, [::]:8444->8444/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp, 0.0.0.0:9443->8443/tcp, [::]:9443->8443/tcp
postgres       Up 3 minutes (healthy)  5432/tcp
vdr            Up 3 minutes
wallet         Up 3 minutes
```

11.3 El recorrido de una emisión y una presentación



💡 Hay **dos pruebas del titular** (una al emitir, otra al presentar), ambas con la clave de `cnf`, y **una firma del emisor** sobre la credencial.

12. El entorno para esta ERSO

12.1 Preparar el terminal

Atajo	Qué hace
<code>CRED</code> <code><escenario></code>	Ejecuta un escenario de la cartera contra el servicio real, con las credenciales del portal de administración (certificado + token)
<code>H</code>	Ejecuta el teléfono simulado
<code>CW</code>	Opciones de <code>curl</code> al canal 8444
<code>PSQL</code>	Entra a la base de datos

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ W=./evidencias/work
$ BASE=https://$D:8444
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ CW="--cacert certs/ca.crt --resolve $D:8444:127.0.0.1"
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ H() { docker compose --progress quiet --profile tools run --rm -T holder "$@"; }
$ WALLET() { H escenario "$1" --authority authority.json --domain $D --base $BASE --ca /certs/ca.crt "$@"; }
$ CRED() { H escenario "$1" --authority authority.json --domain $D --base $BASE --ca /certs/ca.crt --admin-url https://$D:8443 --p12 /certs/vdr-admin.p12 --p12-pass $P12_PASS --admin-token $CREDENTIAL_ADMIN_TOKEN "$@"; }
```

12.2 La puerta de nginx para credenciales

```
$ sed -n '/listen 8444/,/location \\/ { return 404; }/p' nginx/nginx.conf | cut -c1-165
listen 8444 ssl;
server_name civica-desarrollo.avance.org.co localhost;
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;

add_header X-Content-Type-Options nosniff always;
client_max_body_size 256k;

# API del Wallet Backend (gestión de cartera y del DID del titular)
location /wallet/v1/ { set $wallet_upstream http://wallet:8090; proxy_pass $wallet_upstream;
}

# OpenID4VCI 1.0 (emisión) y OpenID4VP 1.0 (presentación): los únicos endpoints de
credenciales
location = /.well-known/openid-credential-issuer { limit_except GET { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_
location = /.well-known/oauth-authorization-server { limit_except GET { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_
location = /token { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_
location = /nonce { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_
location = /credential { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_
location = /verifier/response { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_

location / { return 404; }
```

- Cada `location =` es **exactamente** un endpoint estándar, con el verbo único permitido (`limit_except GET/POST ... { deny all; }`).
- Los portales `/admin/...` **no están** aquí: solo en la puerta de escritura (9443) con certificado.
- `location / { return 404; }`: cualquier otra ruta no existe.

12.3 El inventario de endpoints (en el código)

Cada ruta se declara con su clase y la norma que la justifica (`ep(método, ruta, clase, norma, autenticación)`):

```
$ grep -n 'ep(' ../credential-service/src/main/kotlin/co/org/avance/ssi/credential/Server.kt
| sed -E 's/^[0-9]+: *//; s/ \{.*$//'
ep("GET", "/health", "OPS", "n/a", "ninguna")
ep("GET", "/.well-known/openid-credential-issuer", "OID4VCI", "OpenID4VCI 1.0 §12.2", "ninguna")
ep("GET", "/.well-known/oauth-authorization-server", "OID4VCI", "RFC 8414 (usado por OpenID4VCI 1.0 §11)", "ninguna")
ep("POST", "/token", "OID4VCI", "RFC 6749 + OpenID4VCI 1.0 §6 (pre-authorized_code)", "código pre-
autorizado (+ tx_code)")
ep("POST", "/nonce", "OID4VCI", "OpenID4VCI 1.0 §7", "ninguna")
ep("POST", "/credential", "OID4VCI", "OpenID4VCI 1.0 §8", "token de acceso + prueba de posesión")
ep("POST", "/verifier/response", "OID4VP", "OpenID4VP 1.0 §8.2 (response_mode=direct_post)", "state +
nonce de la solicitud")
ep("POST", "/admin/offers", "ADMIN", "Portal del emisor (no estandarizado); produce una oferta
OpenID4VCI 1.0 §4", "token de administración")
ep("POST", "/admin/verifier/requests", "ADMIN", "Portal del verificador (no estandarizado); produce
una solicitud OpenID4VP 1.0 §5", "token de administración")
ep("GET", "/admin/verifier/requests/{id}", "ADMIN", "Portal del verificador (no estandarizado)",
"token de administración")
```

```
ep("GET", "/admin/execution-log", "OPS", "Evidencia del criterio 2 (registro de ejecución)", "token de administración")
```

Clases: 0ID4VCI (emisión), 0ID4VP (presentación), ADMIN (portales, **no** son protocolo de credenciales), OPS (salud y evidencia).

12.4 El perfil en el código

```
$ sed -n '/object CredentialProfiles/,/^}/p' ../credentials-core/src/main/kotlin/co/org/avance/ssi/credentials/Profiles.kt | cut -c1-190
object CredentialProfiles {
    const val FORMAT_SD_JWT_VC = "dc+sd-jwt"
    const val FORMAT_MDOC = "mso_mdoc"
    const val TYP_SD_JWT_VC = "dc+sd-jwt"
    const val TYP_SD_JWT_VC_LEGACY = "vc+sd-jwt"
    const val TYP_KEY_BINDING = "kb+jwt"
    const val TYP_PROOF_OF_POSSESSION = "openid4vci-proof+jwt"
    const val JOSE_ALG = "ES256"
    const val COSE_ALG_ES256 = -7
    const val SD_ALG = "sha-256"
    const val MDOC_DIGEST_ALG = "SHA-256"

    data class FormatProfile(
        val format: String,
        val artifact: String,
        val signatureAlgorithms: List<String>,
        val digestAlgorithm: String,
        val holderProof: String,
        val references: List<String>,
    )

    val declared: List<FormatProfile> = listOf(
        FormatProfile(
            FORMAT_SD_JWT_VC, "SD-JWT VC (JWS compacto + divulgaciones), typ=$TYP_SD_JWT_VC, prueba
del titular = KB-JWT typ=$TYP_KEY_BINDING",
            listOf(JOSE_ALG), SD_ALG, "cnf.jwk (EC P-256) + KB-JWT ES256 con sd_hash, aud y nonce",
            listOf("draft-ietf-oauth-sd-jwt-vc", "RFC 9901 (SD-JWT) o su borrador vigente",
"OpenID4VCI 1.0 Anexo A.3"),
        ),
        FormatProfile(
            FORMAT_MDOC, "mdoc IssuerSigned (CBOR) con MSO firmado en COSE_Sign1",
            listOf("ES256 (COSE alg=$COSE_ALG_ES256)", MDOC_DIGEST_ALG, "deviceKeyInfo.deviceKey
(COSE_Key EC2 P-256) en el MSO",
            listOf("ISO/IEC 18013-5", "OpenID4VCI 1.0 Anexo A.2"),
        ),
    )

    /** Protocolos permitidos: NO se definen protocolos propios paralelos (criterio 3). */
    val PROTOCOLS = listOf("OpenID4VCI 1.0", "OpenID4VP 1.0")
}
```

Esta es **la fuente única**: el documento de perfil, los metadatos del emisor y el registro de ejecución se derivan de aquí. Dos formatos, un único algoritmo de firma (ES256 / COSE -7) y un resumen (sha-256 / SHA-256).

12.5 Dependencias del servicio

```
$ grep -n 'implementation(' ../credentials-core/build.gradle.kts ../credential-
service/build.gradle.kts | cut -c1-150
../credential-service/build.gradle.kts:12: implementation(project(":did-core"))
../credential-service/build.gradle.kts:13: implementation(project(":did-resolver"))
../credential-service/build.gradle.kts:14: implementation(project(":credentials-core"))
```

```

../credential-service/build.gradle.kts:15: implementation(libs.ktor.server.core)
../credential-service/build.gradle.kts:16: implementation(libs.ktor.server.netty)
../credential-service/build.gradle.kts:17: implementation(libs.ktor.server.status.pages)
../credential-service/build.gradle.kts:18: implementation(libs.ktor.client.core)
../credential-service/build.gradle.kts:19: implementation(libs.ktor.client.java)
../credential-service/build.gradle.kts:20: implementation(libs.kotlinox.coroutines.core)
../credential-service/build.gradle.kts:21: implementation(libs.logback)

```

El servicio de credenciales usa `did-core` y `did-resolver` (para encontrar la clave del emisor por su DID) y `credentials-core` (los formatos). **No usa** ninguna biblioteca de credenciales de terceros.

13. Mapa de requisitos a implementación

Paso	Implementación	Dónde
1 · Perfil	<code>CredentialProfiles</code> + <code>PERFIL-CREDENCIALES.md</code> ; metadatos del emisor lo anuncian	<code>Profiles.kt</code>
2 · Emisión <code>dc+sd-jwt</code>	<code>SdJwtVcIssuer</code> (<code>typ dc+sd-jwt</code> , <code>_sd_alg=sha-256</code> , <code>cnf</code> , <code>ES256</code>)	<code>SdJwtVc.kt</code> , <code>IssuerService.kt</code>
3 · Recepción <code>mso_mdoc</code>	<code>MdocVerifier</code> : firma COSE ES256, resúmenes, vigencia, <code>deviceKey</code>	<code>Mdoc.kt</code> , <code>CredentialWallet.kt</code>
4 · Separación de firmas	Dos claves, dos verificaciones; <code>KeyMatrix</code> falla si comparten clave	<code>KeyMatrix.kt</code>
5 · Diferencias de bibliotecas	No se reutilizó ninguna; diferencias con EUDI documentadas	<code>docs/ANALISIS-BASES-EUDI.md</code>
6 · Sin protocolo propio	<code>EndpointCatalog</code> ; prueba de inventario y sondeo	<code>Server.kt</code> , <code>Erso002CredentialsTest</code> > C3

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO

14. Cómo funcionan los laboratorios

Cada laboratorio:  **predicción** → **comandos** (con su respuesta real y su explicación) →  **conclusión**.

El laboratorio central es **el recorrido** (`CRED tour-credentials`): se ejecuta **una vez** (laboratorio B) y su salida se reparte entre los criterios 1, 2 y 4. Al comienzo de cada bloque verás el mismo comando, y un comentario `# ... (extracto...)` que indica qué sección del recorrido se muestra.

Lab	Criterio	Qué se prueba	Sección
A	Paso 1	El perfil por formato y los algoritmos	\$16
B	1	Emisión y recepción en <code>dc+sd-jwt</code> y <code>mso_mdoc</code>	\$17
B	2	Declarado = usado	\$18
D	3	Ningún protocolo propio	\$19
B	4	Firma del emisor separada de la prueba del titular	\$20
E	—	Pruebas automáticas	\$21
F	—	Recorrido de extremo a extremo	\$22

15. Preparación

Ver §12.1. El recorrido usa el portal de administración (por el canal de escritura `9443`, con certificado y token) para crear la oferta y la solicitud, y la cartera simulada para canjear y presentar.

16. Laboratorio A — Paso 1: el perfil por formato

Qué se busca. Leer el **documento de perfil** que declara qué formatos y algoritmos se admiten.

```
$ sed -n '1,45p' ../docs/PERFIL-CREDENCIALES.md | cut -c1-200
# Perfil por formato de credencial (ERSo 2026-002, paso 1)

Documento de perfil: **lo que el proyecto declara**. La fuente única en código es `credentials-core/.../Profiles.kt` (`CredentialProfiles`); un registro de ejecución (`ExecutionLog`) lee los artefa

> Los detalles marcados con  están escritos de memoria del texto de las especificaciones y **deben confirmarse** contra la versión exacta adoptada por el equipo (ver `VERSIONES-NORMATIVAS.md`)
```

1. Formatos y algoritmos admitidos

	Formato	Algoritmo	Cabecera	Prueba
	`dc+sd-jwt` (SD-JWT VC) `mso_mdoc` (mdoc)			
	Artefacto JWS compacto + divulgaciones: `jwt-d1~d2~[kb-jwt]` CBOR `IssuerSigned` (`nameSpaces` + `issuerAuth` = COSE_Sign1 con el MSO)			
	Algoritmo de firma **ES256** (ECDSA P-256 + SHA-256) **ES256** (COSE `alg` = -7)			
	Cabecera `typ` = dc+sd-jwt, `alg` = ES256, `kid` = <DID>#key-1 protegida `{1: -7}`; no protegida `{4: kid}`			
	Digest de divulgación `_sd_alg` = sha-256 `digestAlgorithm` = SHA-256			
	Clave del titular `cnf.jwk` (EC P-256, sin `d`) `deviceKeyInfo.deviceKey` (COSE_Key EC2 P-256)			
	Prueba del titular al presentar KB-JWT `typ` = kb+jwt, ES256, `{iat, aud, nonce, sd_hash}` **no implementada** (DeviceResponse)			
	Identificador del formato en OpenID4VCI `dc+sd-jwt` `mso_mdoc`			
	Tipo de credencial `vct` = urn:avance:credential:academic:1 `docType` = org.avance.academic.1			

Todo lo demás se **rechaza**: otros algoritmos (ES384, EdDSA, RS256...), `typ` = vc+sd-jwt (perfil anterior; solo con `acceptLegacyTyp=true`), `\_sd\_alg` distinto de sha-256.

2. Protocolos (sin protocolos propios)

- * Emisión: **OpenID4VCI 1.0**, flujo de **código pre-autorizado** (con `tx\_code` opcional). Endpoints: metadatos del emisor, metadatos del servidor de autorización, `token`, `nonce`, `credential`.
- * Presentación: **OpenID4VP 1.0**, `response\_mode` = direct_post, prefijo de cliente `redirect\_uri`, consulta **DCQL**. ⚠
- * Prueba de posesión en la emisión: `proofs: { jwt: [ ... ] }`, JWT `typ` = openid4vci-proof+jwt, `alg` = ES256, clave pública en la cabecera `jwk`, `aud` = identificador del emisor, `nonce` = `c\_n`
- * Respuesta de credencial: `{ "credentials": [ { "credential": "..." } ] }`; en `mso\_mdoc`, el `IssuerSigned` en **base64url**. ⚠

3. Roles de clave (matriz)

	Clave	Vive en	Firma	Se publica en
	Emisor (`ISSUER_SIGNING`) Signature Service del emisor credencial (JWT y COSE_Sign1) DID Document del emisor en el VDR (`assertionMethod`)			
	Titular (`HOLDER_PROOF`) Hardware seguro del dispositivo prueba de posesión (JWT de OpenID4VCI, KB-JWT) dentro de la credencial (`cnf` / `deviceKeyInfo`)			

Las dos verificaciones son independientes: la clave del titular no valida la firma del emisor y viceversa (`Erso002CredentialsTest > C4`).

4. Desviaciones declaradas respecto a los perfiles de referencia

	Tema	Perfil de referencia (EUDI / ISO)	Este proyecto	Consecuencia
	Clave del emisor `x5c` (SD-JWT) / `x5chain` (mdoc) `kid` = URL DID resuelta en el VDR Un verificador EUDI/ISO estándar no valida estas credenciales sin adaptación			
	Prueba de posesión `jwt` con key attestation, o `attestation` `jwt` sin key attestation El emisor no comprueba el hardware de la clave del titular al emitir			
	Cifrado JWE Exigido por algunos emisores No Metadatos y respuestas en claro sobre TLS			
	Token AS externo (Keycloak) + DPoP AS propio del emisor, sin DPoP Tokens de portador			

✅ **Cómo leerlo.** La tabla de §1 del documento es el **contrato**: para cada formato, el artefacto, el algoritmo (siempre ES256), el tipo de cabecera, el resumen, dónde va la clave del titular y cómo prueba el titular al presentar. Y una línea clave: «**Todo lo demás se rechaza**» (otros algoritmos, `typ` = vc+sd-jwt`, otro `_sd_alg``). Observa también una confesión honesta: la presentación de mdoc **no está implementada**.

17. CRITERIO 1 — Emisión y recepción en `dc+sd-jwt` y `mso_mdoc`

Criterio. "La emisión y la recepción funcionan en `dc+sd-jwt` y `mso_mdoc` conforme al perfil adoptado; evidencia: pruebas cruzadas con `OpenID4VCI 1.0` y `OpenID4VP 1.0`."

Predice antes de ejecutar

(1) Una credencial SD-JWT tiene 4 datos personales (nombre, apellido, programa, promedio). ¿Dónde están dentro del JWT firmado? (2) Cuando la titular presente solo el programa, ¿qué verá el verificador de los otros tres?

17.1 Ejecutar el recorrido

El recorrido completo se ejecuta aquí; a continuación, sección por sección.

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 1; el resto del recorrido se muestra en las demás secciones)
— 1. Metadatos del emisor (OpenID4VCI 1.0)
{
  "credential_issuer": "https://civica-desarrollo.avance.org.co:8444",
  "credential_endpoint": "https://civica-desarrollo.avance.org.co:8444/credential",
  "nonce_endpoint": "https://civica-desarrollo.avance.org.co:8444/nonce",
  "credential_configurations_supported": {
    "AcademicCredential_dc+sd-jwt": {
      "format": "dc+sd-jwt",
      "vct": "urn:avance:credential:academic:1",
      "scope": "academic_sdjwt",
      "cryptographic_binding_methods_supported": [
        "jwk"
      ],
      "credential_signing_alg_values_supported": [
        "ES256"
      ],
      "proof_types_supported": {
        "jwt": {
          "proof_signing_alg_values_supported": [
            "ES256"
          ]
        }
      },
      "credential_metadata": {
        "display": [
          {
            "name": "Credencial académica",
            "locale": "es"
          }
        ]
      }
    },
    "AcademicCredential_mso_mdoc": {
      "format": "mso_mdoc",
      "doctype": "org.avance.academic.1",
      "scope": "academic_mdoc",
      "cryptographic_binding_methods_supported": [
        "cose_key"
      ],
      "credential_signing_alg_values_supported": [
        -7
      ]
    }
  }
}
```

```

    "proof_types_supported": {
      "jwt": {
        "proof_signing_alg_values_supported": [
          "ES256"
        ]
      }
    },
    "credential_metadata": {
      "display": [
        {
          "name": "Credencial académica (mdoc)",
          "locale": "es"
        }
      ]
    }
  }
}
}
}

```

- **Qué hace.** Pide los metadatos del emisor (/.well-known/openid-credential-issuer).
- **Qué significa.** El emisor **anuncia** dos configuraciones: AcademicCredential_dc+sd-jwt (con vct, enlace por jwk, firma ES256) y AcademicCredential_mso_mdoc (con doctype, enlace por cose_key, firma -7). En ambas: la prueba de posesión del titular admitida es jwt con ES256. Es OpenID4VCI 1.0 §12.2.

17.2 La oferta

```

$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 2; el resto del recorrido se muestra en las demás secciones)
— 2. Oferta creada en el portal del emisor (HTTP 201)
{
  "credential_offer": {
    "credential_issuer": "https://civica-desarrollo.avance.org.co:8444",
    "credential_configuration_ids": [
      "AcademicCredential_dc+sd-jwt",
      "AcademicCredential_mso_mdoc"
    ],
    "grants": {
      "urn:iETF:params:oauth:grant-type:pre-authorized_code": {
        "pre-authorized_code": "K1sgwtLPOLSmAvL_sbTZ2Qf0ztftWETeajLwZqtHPkk",
        "tx_code": {
          "input_mode": "numeric",
          "length": 6
        }
      }
    }
  },
  "credential_offer_uri_by_value": "openid-credential-offer://?
credential_offer=%7B%22credential_issuer%22%3A%22https%3A%2F%2Fcivica-
desarrollo.avance.org.co%3A8444%22%2C%22credential_configuration_ids%22%3A%5B%22AcademicCredential_dc
%2Bsd-
jwt%22%2C%22AcademicCredential_mso_mdoc%22%5D%2C%22grants%22%3A%7B%22urn%3Aietf%3Aparams%3Aoauth%3Agr
ant-type%3Apre-authorized_code%22%3A%7B%22pre-
authorized_code%22%3A%22K1sgwtLPOLSmAvL_sbTZ2Qf0ztftWETeajLwZqtHPkk%22%2C%22tx_code%22%3A%7B%22input_
mode%22%3A%22numeric%22%2C%22length%22%3A%6%7D%7D%7D%7D",
  "tx_code": "876214"
}
canje de la oferta (null = sin errores): null

```

- **Qué hace.** El **portal del emisor** (administración) crea una oferta con los datos de Ana.
- **Qué significa.** La oferta incluye las dos configuraciones, el **código pre-autorizado** y la indicación de que se requiere un tx_code (numérico, 6 dígitos), que el portal devuelve

aparte (`tx_code`: se entregaría al titular por otro canal). El portal es de administración y **no** es un protocolo de credenciales.

17.3 La credencial `dc+sd-jwt`, por partes

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 3; el resto del recorrido se muestra en las demás secciones)
— 3. Credencial dc+sd-jwt recibida y verificada por la cartera
  Estructura: JWT~divulgación~divulgación~...~ (4 divulgaciones, sin KB-JWT)
  Cabecera del JWT:
  {
    "alg": "ES256",
    "typ": "dc+sd-jwt",
    "kid": "did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1"
  }
  Contenido del JWT:
  {
    "iss": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
    "iat": 1790882204,
    "exp": 1793474204,
    "vct": "urn:avance:credential:academic:1",
    "cnf": {
      "jwk": {
        "kty": "EC",
        "crv": "P-256",
        "x": "zAu76lp1ytLCPkgbh0k3w9x3E8nV4uZydkvwVYC-Eko",
        "y": "Cx14luhbxB-crfuNoxCWoUDF0PSTTTKkBnbwPiellM"
      }
    },
    "_sd": [
      "dtDFmt_-bayNWNxCMKpizhjyb9Xfpj41co8cdwGePKw",
      "e67sjZsVu86zgQofmNdXjFwyJd7y9fo4S00D48yPpcA",
      "lBF7-8ZQ_dqq4NiSSpXeF56kG7ZPI3kx0vmAExNC6U",
      "w9TgAUEit0DGPImh0AADi40KePoiqLgJa9VdkxLqWtE"
    ],
    "_sd_alg": "sha-256"
  }
  Divulgaciones (cada una = [sal, nombre, valor]) y su digest en `_sd`:
  ["VVtj9MC3WJHWHu7dWdK4Qg", "given_name", "Ana"] digest=dtDFmt_-bayNWNxC... ¿en_sd?=true
  ["Mx2GaL0uEinP-osJczykyA", "family_name", "Pérez"] digest=lBF7-8ZQ_dqq4NiS... ¿en_sd?=true
  ["BQA1pY_8QWCpMFt8hU2CRg", "program", "Ingeniería de Sistemas"] digest=w9TgAUEit0DGPImh... ¿en_sd?=true
  ["MWQTK_SihSlTfyCmrj4T3A", "gpa", "4.5"] digest=e67sjZsVu86zgQof... ¿en_sd?=true
  huella de cnf.jwk (clave del titular): Llssu_eZPCUKS0eaS307PNlfQ1v2E3rA6mWLOtEFw-g
  huella de la clave del titular en el custodio: Llssu_eZPCUKS0eaS307PNlfQ1v2E3rA6mWLOtEFw-g
```

✓ Cómo leerlo.

Parte	Qué es	Qué observar
Estructura	JWT~divulgación~divulgación~...~	Cuatro divulgaciones, sin KB-JWT (aún no se presenta)
Cabecera	alg: ES256, typ: dc+sd-jwt, kid: <DID del emisor>#key-1	El kid es una DID URL : así el verificador sabe dónde buscar la clave pública
Contenido	iss (el DID del emisor), iat, exp, vct, cnf.jwk (clave pública del titular), _sd (4 resúmenes) y _sd_alg: sha-256	No aparece ningún dato personal. Solo cuatro resúmenes
Divulgaciones	["sal", "given_name", "Ana"], ...	Cada una con su resumen, y la comprobación ¿en_sd? = true

Parte	Qué es	Qué observar
<code>cnf</code>	Clave pública del titular	Su huella coincide con la de la clave del titular en el custodio

Es la respuesta a la predicción (1): los datos personales **no están** en el JWT firmado; están **ocultos tras resúmenes** que sí van firmados. La titular guarda las divulgaciones aparte y decide cuáles incluir.

17.4 La credencial `mso_mdoc`

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 4; el resto del recorrido se muestra en las demás secciones)
— 4. Credencial mso_mdoc recibida y verificada
  docType           : org.avance.academic.1
  issuerKid         : did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1
  elementos         : {org.avance.academic.1={given_name=Ana, family_name=Pérez,
program=Ingeniería de Sistemas, gpa=4.5}}
  huella de deviceKey : Llssu_eZPCUkS0eaS307PNlfQ1v2E3rA6mWL0tEFw-g
  tamaño IssuerSigned (bytes CBOR) : 1031
```

- **Qué significa.** La cartera recibió el binario CBOR (`IssuerSigned`, 1031 bytes), **verificó** el MSO (firma COSE ES256, resúmenes de cada elemento y vigencia) y lo guardó. Se ven el `docType`, el `issuerKid` (otra vez el DID del emisor), los elementos y la huella de `deviceKey` —la misma que la `cnf` del SD-JWT—: ambas credenciales quedan ligadas a la **misma clave del titular**.

17.5 La cartera no confía en lo recibido

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 5; el resto del recorrido se muestra en las demás secciones)
— 5. Manipulaciones: la cartera NO confía en lo recibido
  (a) cambiar el `vct` del contenido firmado: rechazada: La firma del emisor no es válida
  (b) cambiar una divulgación (gpa 4.5 → 5.0): rechazada: Divulgación sin digest en la credencial:
'gpa'
  tx_code incorrecto           : 400 invalid_grant
  oferta con el tx_code correcto después del fallo: emitida
```

- **(a) Cambiar el `vct`** del contenido firmado: «La firma del emisor no es válida».
- **(b) Cambiar una divulgación** (`gpa` 4.5 → 5.0): «Divulgación sin digest en la credencial». El resumen de la nueva divulgación no está en `_sd`.
- **`tx_code` incorrecto:** `400 invalid_grant`. Y después, con el correcto, la oferta **sigue disponible** y se emite.

17.6 La presentación (OpenID4VP) con divulgación selectiva

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 6; el resto del recorrido se muestra en las demás secciones)
— 6. Solicitud de presentación (OpenID4VP 1.0, DCQL) y presentación con divulgación selectiva
{
  "client_id": "redirect_uri:https://civica-desarrollo.avance.org.co:8444/verifier/response",
  "response_type": "vp_token",
  "response_mode": "direct_post",
  "response_uri": "https://civica-desarrollo.avance.org.co:8444/verifier/response",
  "nonce": "n6MvweP3sWw6DGPajYah320-M-CVevg9",
  "state": "s4GP4GY9rTwoP2_oT2hZjNq0MeCzqv5h",
```

```

"dcql_query": {
  "credentials": [
    {
      "id": "academic",
      "format": "dc+sd-jwt",
      "meta": {
        "vct_values": [
          "urn:avance:credential:academic:1"
        ]
      },
      "claims": [
        {
          "path": [
            "program"
          ]
        }
      ]
    }
  ]
}
}
divulgaciones presentadas      : [["BQA1pY_8QWCpMFt8hU2CRg","program","Ingeniería de Sistemas"]]
Cabecera del KB-JWT:
{
  "alg": "ES256",
  "typ": "kb+jwt"
}
Contenido del KB-JWT:
{
  "iat": 1790882205,
  "aud": "redirect_uri:https://civica-desarrollo.avance.org.co:8444/verifier/response",
  "nonce": "n6MvweP3sWw6DGPajYah320-M-CVevg9",
  "sd_hash": "ZWG8IYGlow8pQpYql6PbbkGBgHaKy0JoDLIn3oBXhTw"
}
sd_hash recalculado           : ZWG8IYGlow8pQpYql6PbbkGBgHaKy0JoDLIn3oBXhTw
el verificador respondió      : HTTP 200 {}

```

✓ Cómo leerlo.

- **Solicitud:** `client_id` (el verificador), `nonce` y `state` (de un solo uso), `response_mode: direct_post` y `dcql_query` («una credencial `dc+sd-jwt` con `vct` académico, y quiero el dato `program`»).
- **Divulgaciones presentadas:** solo `program`.
- **KB-JWT:** `typ: kb+jwt`; contenido con `iat`, `aud` (el verificador), `nonce` (el de la solicitud) y `sd_hash`. El `sd_hash` recalculado coincide: cubre exactamente lo presentado.
- El verificador respondió `HTTP 200`.

17.7 Lo que ve el verificador

```

$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 7; el resto del recorrido se muestra en las demás secciones)
— 7. Resultado en el verificador
{
  "requestId": "-kAk5xAw_LTcXUaz0vHNwxPdC8xIho_U",
  "status": "VERIFIED",
  "reason": null,
  "issuer": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
  "claims": {
    "iss": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
    "vct": "urn:avance:credential:academic:1",
    "program": "Ingeniería de Sistemas"
  }
}

```

```
}
  repetir la misma respuesta      : HTTP 400 {"error":"invalid_request","error_description":"La
solicitud ya fue respondida (VERIFIED)"}
}
```

- **Qué significa.** status: VERIFIED, el emisor, y **solo** tres claims: iss, vct y program. El nombre y el promedio **no llegaron**. Respuesta a la predicción (2). Repetir la misma respuesta se rechaza (La solicitud ya fue respondida).

Conclusión — evidencia del criterio 1

Evidencia	Resultado
Metadatos OpenID4VCI 1.0 anuncian ambos formatos con ES256	✓
Oferta con código pre-autorizado y tx_code	✓
Emisión dc+sd-jwt verificada por la cartera	✓
Emisión/recepción mso_mdoc verificada	✓
Presentación OpenID4VP con divulgación selectiva	✓
Repetición, tx_code erróneo, credencial alterada → rechazo	✓
Interoperabilidad con implementaciones de terceros	⚠ no demostrada (§27)

18. CRITERIO 2 — Los algoritmos y formatos declarados coinciden con los efectivamente usados

Criterio. "Los algoritmos y formatos declarados coinciden con los efectivamente usados; evidencia: documento de perfil por formato y registro de ejecución."

Predice antes de ejecutar

Si el registro de ejecución lee los artefactos **reales** que produjo el sistema, ¿qué pasaría si algún componente firmara con ES384 sin declararlo?

18.1 El registro de ejecución

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 8; el resto del recorrido se muestra en las demás secciones)
— 8. Registro de ejecución (perfil declarado vs. usado)
{
  "observed": [
    {
      "role": "sd-jwt-vc emitido",
      "format": "dc+sd-jwt",
      "typ": "dc+sd-jwt",
      "alg": "ES256",
      "digest": "sha-256"
    },
    {
      "role": "mso_mdoc emitido",
      "format": "mso_mdoc",
      "typ": null,
      "alg": "ES256",

```

```
    "digest": "SHA-256"
  },
  {
    "role": "sd-jwt-vc emitido",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "sd-jwt-vc prueba del titular (kb-jwt)",
    "format": "dc+sd-jwt",
    "typ": "kb+jwt",
    "alg": "ES256",
    "digest": null
  },
  {
    "role": "sd-jwt-vc emisor",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "sd-jwt-vc emitido",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "mso_mdoc emitido",
    "format": "mso_mdoc",
    "typ": null,
    "alg": "ES256",
    "digest": "SHA-256"
  },
  {
    "role": "sd-jwt-vc emitido",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "sd-jwt-vc prueba del titular (kb-jwt)",
    "format": "dc+sd-jwt",
    "typ": "kb+jwt",
    "alg": "ES256",
    "digest": null
  },
  {
    "role": "sd-jwt-vc emisor",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "sd-jwt-vc emitido",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "mso_mdoc emitido",
```

```

    "format": "mso_mdoc",
    "typ": null,
    "alg": "ES256",
    "digest": "SHA-256"
  },
  {
    "role": "sd-jwt-vc emitido",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  },
  {
    "role": "sd-jwt-vc prueba del titular (kb-jwt)",
    "format": "dc+sd-jwt",
    "typ": "kb+jwt",
    "alg": "ES256",
    "digest": null
  },
  {
    "role": "sd-jwt-vc emisor",
    "format": "dc+sd-jwt",
    "typ": "dc+sd-jwt",
    "alg": "ES256",
    "digest": "sha-256"
  }
],
"mismatches": []
}

```

- **Qué hace.** Consulta `GET /admin/execution-log` en el servicio.
- **Qué significa.** Cada fila es **un artefacto realmente producido o verificado**: su rol (`sd-jwt-vc emitido`, `mso_mdoc emitido`, `prueba del titular (kb-jwt)`, `emisor...`), su formato, su `typ`, su algoritmo y su resumen. Todo es `ES256` y `sha-256 / SHA-256`. Y el campo clave: `"mismatches": []`: ninguna diferencia respecto al perfil declarado.
- **Cómo se llena:** el registro **lee los artefactos** (cabecera, algoritmo) en el momento de emitir y verificar; no copia lo declarado.

18.2 La declaración, otra vez

El documento de perfil (§16) y `CredentialProfiles` (§12.4) declaran: `dc+sd-jwt` y `mso_mdoc`, **solo ES256**, `sha-256 / SHA-256`. Compara con las filas de arriba: coinciden.

18.3 ¿El detector detecta?

Una comprobación sin caso negativo no prueba nada. La prueba `CredentialsCoreTest > criterio 2` - el registro de ejecucion detecta un algoritmo distinto al declarado alimenta el registro con un artefacto `ES384` (o un formato no declarado) y comprueba que **aparece una diferencia**. Está en la lista de §21.

Conclusión — evidencia del criterio 2

Evidencia	Resultado
Documento de perfil por formato	✅ docs/PERFIL-CREDENCIALES.md + Profiles.kt
Registro de ejecución sin diferencias	✅ mismatches: []
El registro detecta lo no declarado	✅ prueba automática

19. CRITERIO 3 — No existe ningún protocolo propio paralelo

Criterio. "No existe ningún protocolo propio paralelo a OpenID4VCI ni OpenID4VP; evidencia: inventario de endpoints y revisión de código de integración."

Predice antes de ejecutar

Si alguien probara rutas típicas de un protocolo inventado (/issue , /present , /api/vc ...), ¿qué respondería la puerta de credenciales? ¿Y el portal de administración, accesible desde el canal público?

19.1 El inventario

El inventario de §12.3 es la evidencia documental: **11 endpoints**, cada uno con su clase y norma. Los de clase 0ID4VCI y 0ID4VP citan secciones de OpenID4VCI/VP o de los RFC en que se apoyan (6749, 8414). Los ADMIN son portales de uso interno, no un protocolo de credenciales; los OPS , salud y evidencia.

19.2 El sondeo de rutas de un posible protocolo propio

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 11; el resto del recorrido se muestra en las demás secciones)
— 11. Sondeo de rutas de un posible protocolo propio
GET /issue -> 404
GET /credentials -> 404
GET /present -> 404
GET /presentation -> 404
GET /api/vc -> 404
GET /vc -> 404
GET /vp -> 404
GET /oidc4vc -> 404
GET /verify -> 404
POST /admin/offers (canal público 8444) -> 404
· 1 claves privadas de prueba registradas en secrets-cred.txt (solo laboratorio) para buscarlas en
la base de datos
RESULTADO tour-credentials: 0 PASS · 0 FAIL
```

- **Qué hace.** Pide 9 rutas típicas de un protocolo inventado y prueba el portal de administración por el canal público.
- **Qué significa.** Todas 404 . Ninguna ruta propia existe.

19.3 El portal de administración no está en el canal público

```
$ curl -s -o /dev/null -w "POST /admin/offers por el canal público 8444 -> HTTP %{http_code}\n" $CW -X POST $BASE/admin/offers
$ curl -s -o /dev/null -w "POST /admin/offers por 9443 con certificado pero SIN token -> HTTP %{http_code}\n" --cacert certs/ca.crt --resolve $D:9443:127.0.0.1 --cert certs/vdr-admin.crt --key certs/vdr-admin.key -X POST https://$D:9443/admin/offers -H "Content-Type: application/json" -d '{} '
$ curl -s -o /dev/null -w "POST /admin/offers por 9443 SIN certificado -> HTTP %{http_code}\n" --cacert certs/ca.crt --resolve $D:9443:127.0.0.1 -X POST https://$D:9443/admin/offers -d '{} '
$ curl -s -o /dev/null -w "GET /token (verbo equivocado) por 8444 -> HTTP %{http_code}\n" $CW $BASE/token
POST /admin/offers por el canal público 8444 -> HTTP 404
POST /admin/offers por 9443 con certificado pero SIN token -> HTTP 401
```

```
POST /admin/offers por 9443 SIN certificado -> HTTP 400
GET /token (verbo equivocado) por 8444 -> HTTP 403
```

- **Qué significa.**

- Por el canal público (8444): 404 . El portal **no existe** allí.
- Por 9443 con certificado, **sin token**: 401 . Existe, pero exige token de administración.
- Por 9443 **sin certificado**: 400 de nginx.
- GET /token (el endpoint es POST): 403 . nginx permite **un solo verbo** por ruta.

19.4 Los metadatos del servidor de autorización

```
$ curl -s $CW $BASE/.well-known/oauth-authorization-server | jq -c '{issuer, token_endpoint, grant_types_supported}'
{"issuer":"https://civica-desarrollo.avance.org.co:8444","token_endpoint":"https://civica-desarrollo.avance.org.co:8444/token","grant_types_supported":["urn:ietf:params:oauth:grant-type:pre-authorized_code"]}
```

La emisión usa un único flujo: el **código pre-autorizado** (urn:ietf:params:oauth:grant-type:pre-authorized_code) con el endpoint /token estándar.

19.5 Revisión del código de integración

El servicio solo registra rutas mediante ep(...) , que añade cada una al **catálogo** (el inventario sale del código, no de un documento escrito a mano). La prueba C3 inventario de endpoints - solo OpenID4VCI, OpenID4VP, administracion y operacion compara el catálogo contra la lista permitida y falla si aparece otra ruta.

🎯 Conclusión — evidencia del criterio 3

Evidencia	Resultado
Inventario con norma por endpoint	✓
9 rutas típicas de protocolo propio	✓ todas 404
Portal de administración fuera del canal público	✓ 404
nginx: una ruta, un verbo	✓
Prueba de inventario	✓

20. CRITERIO 4 — La firma del emisor está separada de la prueba del titular

Criterio. "La firma del emisor está separada de la prueba del titular; evidencia: matriz de claves por rol y pruebas de verificación independientes."

✍ Predice antes de ejecutar

Si verifico la firma de la **credencial** con la clave del **titular**, ¿qué resultado esperas? ¿Y la firma del **KB-JWT** con la clave del **emisor**?

20.1 La matriz de claves por rol

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 9; el resto del recorrido se muestra en las demás secciones)
— 9. Matriz de claves por rol
| Rol | Huella (RFC 7638) | Algoritmo | La clave privada vive en | Firma | Clave pública publicada en |
| Verifica |
| ---|---|---|---|---|---|
| ISSUER_SIGNING | `chVecCDfdMjvjkWVer5aaklRDA-9kyWkClX_QCxGTkA` | ES256 (JOSE) / ES256 (COSE alg=-7) |
| Signature Service del emisor (nunca sale del servicio) | la credencial: JWT del SD-JWT VC y
COSE_Sign1 del MSO | DID Document del emisor (did:web:civica-
desarrollo.avance.org.co:entidades:avance#key-1) en el VDR | cualquier verificador, resolviendo el
DID del emisor |
| HOLDER_PROOF | `Llssu_eZPCUkS0eaS307PNlfQ1v2E3rA6mWL0tEFw-g` | ES256 | Hardware seguro del
dispositivo del titular (no exportable) | la prueba de posesión: JWT de prueba de OpenID4VCI y KB-JWT
de la presentación | dentro de la credencial: 'cnf.jwk' (SD-JWT) o 'deviceKeyInfo' (mdoc) | emisor
(al emitir) y verificador (al presentar), con la clave de 'cnf' |
```

✓ **Cómo leerla.** Dos filas:

Rol	Huella	Dónde vive la clave privada	Qué firma	Dónde se publica la pública	Quién la verifica
ISSUER_SIGNING	chVecCDf...	Servicio de firma del emisor	La credencial (JWT del SD-JWT VC y COSE_Sign1 del MSO)	DID Document del emisor (VDR)	Cualquier verificador, resolviendo el DID
HOLDER_PROOF	Llssu_eZ...	Hardware seguro del dispositivo	La prueba de posesión (JWT de emisión y KB-JWT)	Dentro de la credencial (cnf / deviceKey)	El emisor y el verificador

Las huellas son **distintas**. La matriz sale de las **claves reales** de la ejecución, no de un texto escrito a mano.

20.2 Verificaciones independientes

```
$ CRED tour-credentials --secrets-out secrets-cred.txt
# ... (extracto del recorrido: sección 10; el resto del recorrido se muestra en las demás secciones)
— 10. Verificaciones independientes
firma de la credencial con la clave del EMISOR: true
firma de la credencial con la clave del TITULAR: false
firma del KB-JWT con la clave del TITULAR: true
firma del KB-JWT con la clave del EMISOR: false
```

- **Qué hace.** Verifica cuatro combinaciones:

Firma	Clave usada	Resultado
La credencial	del emisor	true
La credencial	del titular	false
El KB-JWT	del titular	true
El KB-JWT	del emisor	false

- **Qué significa.** Cada firma solo se valida con **su** clave. Respuesta a la predicción: `false` en ambos cruces.

20.3 La garantía en el código

La prueba `criterio 4 - matriz de claves por rol y rechazo de claves compartidas` comprueba que `KeyMatrix` **falla** si ambos roles comparten clave; y `sd-jwt` - la firma del emisor no se puede reemplazar por otra clave y `sd-jwt` - la prueba del titular sola no basta - `verificacion independiente de la firma del emisor` prueban los dos sentidos. Se listan en §21.

🔗 Conclusión — evidencia del criterio 4

Evidencia	Resultado
Matriz con dos claves distintas	✓
Cada firma valida solo con su clave	✓ 4 combinaciones
La clave del emisor viene del DID; la del titular, de <code>cnf/deviceKey</code>	✓
El código rechaza claves compartidas	✓ prueba

21. Pruebas automáticas

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :credentials-core:test
:credential-service:test --rerun-tasks -q 2>&1 | tail -3; echo '(sin salida = todas pasaron)'
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> for pat in ['credentials-core/build/test-results/test/*CredentialsCoreTest*.xml', 'credential-
service/build/test-results/test/*Erso002CredentialsTest*.xml']:
>     for f in sorted(glob.glob(pat)):
>         s = open(f, encoding='utf-8').read()
>         tot = re.search(r'name="([^\"]+)" tests="(\d+)" skipped="(\d+)" failures="(\d+)"',
s).groups()
>         print(f"{tot[0].split('.')[1]}: {tot[1]} pruebas, {tot[2]} omitidas, {tot[3]} fallos")
>         for m in re.finditer(r'<testcase name="([^\"]+)"[^>]*?(/>|>(.*)</testcase>)', s, re.S):
>             name = html.unescape(m.group(1)).removesuffix("(")
>             estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>             print(f"    [{estado}] {name}")
>         print()
> EOF
CredentialsCoreTest: 12 pruebas, 0 omitidas, 0 fallos
[PASA ] jwk - no acepta material privado ni puntos fuera de la curva
[PASA ] mdoc - un elemento alterado, una firma ajena o un algoritmo distinto se rechazan
[PASA ] criterio 4 - matriz de claves por rol y rechazo de claves compartidas
[PASA ] sd-jwt - una divulgacion alterada o ajena se rechaza
[PASA ] sd-jwt - perfil - typ vc+sd-jwt (perfil anterior) se rechaza salvo habilitarlo
[PASA ] mdoc - emision y recepcion verificada
[PASA ] sd-jwt - prueba del titular (KB-JWT) - clave ajena, audiencia, nonce y repeticion
[PASA ] criterio 2 - el registro de ejecucion detecta un algoritmo distinto al declarado
[PASA ] sd-jwt - la prueba del titular sola no basta - verificacion independiente de la firma del
emisor
[PASA ] sd-jwt - la firma del emisor no se puede reemplazar por otra clave
[PASA ] sd-jwt - emision, divulgacion selectiva y presentacion validas
[PASA ] sd-jwt - expiracion

Erso002CredentialsTest: 16 pruebas, 0 omitidas, 0 fallos
[PASA ] C1 metadatos del emisor OpenID4VCI 1_0 anuncian los dos formatos y el perfil
```

```

[PASA ] C1 el verificador rechaza repeticion, nonce o audiencia ajenos, claims faltantes y formatos no soportados
[PASA ] C1 dc+sd-jwt - emision con oferta y tx_code, recepcion verificada y presentacion con divulgacion selectiva
[PASA ] C1 la oferta vence
[PASA ] C3 inventario de endpoints - solo OpenID4VCI, OpenID4VP, administracion y operacion
[PASA ] C1 una credencial alterada o de otro emisor se rechaza al presentarla
[PASA ] C2 el registro de ejecucion coincide con el perfil declarado y con los metadatos anunciados
[PASA ] C1 una oferta puede pedir los dos formatos y se emiten ambos
[PASA ] guarda - todo metodo de prueba devuelve void para que JUnit lo ejecute
[PASA ] C1 el portal de administracion exige su token
[PASA ] C1 mso_mdoc - el titular recibe, verifica y almacena la credencial
[PASA ] C4 matriz de claves por rol construida con las claves reales de la ejecucion
[PASA ] C4 verificaciones independientes - la del emisor no usa la clave del titular y la del titular no usa la del emisor
[PASA ] C1 el endpoint de credencial exige token y respeta lo autorizado por la oferta
[PASA ] C1 la prueba de posesion es obligatoria y de un solo uso - nonce, audiencia, firma y fresca
[PASA ] C1 el codigo pre-autorizado es de un solo uso, el tx_code protege y 3 fallos queman la oferta

```

- **Qué hace.** Ejecuta las pruebas de `credentials-core` (formatos) y `credential-service` (protocolo, contra una base desechable).
- **Qué significa.** 12 + 16 pruebas, sin fallos. Los prefijos (`C1`, `C2`, `C3`, `criterio 2`, `criterio 4`) indican el criterio.

Prueba	Criterio
Erso002CredentialsTest > C1 ... (varias)	1
CredentialsCoreTest > sd-jwt ..., mdoc ...	1
C2 registro de ejecucion ..., criterio 2 - el registro ... detecta ...	2
C3 inventario de endpoints ...	3
criterio 4 - matriz ..., sd-jwt - la firma del emisor no se puede reemplazar ...	4

22. Recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` ejecuta todo el proyecto (121 comprobaciones). Este es el bloque de la ERSO 002:

```

=====
ERSo 2026-002 · Suite criptográfica y formatos de credencial (dc+sd-jwt y mso_mdoc sobre OpenID4VCI/OpenID4VP)
=====
✓ PASS C1 metadatos OpenID4VCI 1.0: anuncian dc+sd-jwt y mso_mdoc con ES256
✓ PASS el portal de administración (canal mTLS + token) crea una oferta OpenID4VCI
✓ PASS el portal de administración rechaza llamadas sin token (401)
✓ PASS C1 un tx_code incorrecto es rechazado (invalid_grant)
✓ PASS C1 emisión y recepción en dc+sd-jwt y mso_mdoc (la cartera verificó la firma del emisor resolviendo su DID)
✓ PASS C1 mso_mdoc: MSO verificado, docType y elementos leídos
✓ PASS C1 presentación OpenID4VP (direct_post): el verificador la acepta
✓ PASS C1 divulgación selectiva: el verificador recibió solo el programa (no nombre ni promedio)
✓ PASS C1 repetir la misma respuesta se rechaza
✓ PASS C2 registro de ejecución: sin diferencias respecto al perfil declarado
✓ PASS C2 formatos y algoritmos efectivamente usados = declarados (dc+sd-jwt, mso_mdoc, solo ES256)
  · perfiles declarados: dc+sd-jwt=[ES256], mso_mdoc=[ES256 (COSE alg=-7)]
✓ PASS C3 sin protocolo propio: rutas típicas de un protocolo paralelo devuelven 404 (9 sondeadas)

```

```
✓ PASS C3 sin protocolo propio: los únicos endpoints públicos son los de OpenID4VCI/OpenID4VP
✓ PASS C3 el portal de administración NO está en el canal público (404 en 8444)
✓ PASS C4 la clave del emisor (publicada en su DID Document) y la del titular (cnf) son distintas
| Rol | Huella (RFC 7638) | Algoritmo | La clave privada vive en | Firma | Clave pública publicada en
| Verifica |
|---|---|---|---|---|---|
| ISSUER_SIGNING | `chVecCDfdMjvjkwVEr5aaklRDA-9kyWkClX_QCxDtKA` | ES256 (JOSE) / ES256 (COSE alg=-7)
| Signature Service del emisor (nunca sale del servicio) | la cre...
| HOLDER_PROOF | `xbdNgPi5wnAWilfxe8XCemjor6M7fdXXCiSLAjsL07A` | ES256 | Hardware seguro del
dispositivo del titular (no exportable) | la prueba de posesión: JWT de pr...
```

RESULTADO credentials: 15 PASS · 0 FAIL

```
✓ PASS C4 la matriz de claves por rol se generó con las claves reales (ISSUER_SIGNING y
HOLDER_PROOF)
```

PARTE V — PREGUNTAS Y RESPUESTAS

23. Preguntas sobre los conceptos

- 1. ¿Qué diferencia hay entre una credencial SD-JWT VC y un mdoc?** Ambas son credenciales firmadas por el emisor. La SD-JWT VC es **texto** (JWT/JSON) con divulgación selectiva mediante resúmenes; el mdoc es **binario** (CBOR) heredado de la licencia de conducir móvil (ISO 18013-5), con un MSO firmado en COSE. Sirven para lo mismo; el ecosistema usa ambas.
- 2. ¿Cómo logra el SD-JWT la divulgación selectiva?** Los datos personales **no están** en el JWT: en su lugar hay sus **resúmenes** (`_sd`). Cada dato viaja en una «divulgación» aparte [`sal`, `nombre`, `valor`]. Al presentar, el titular incluye solo las divulgaciones que quiere; el verificador recalcula el resumen de cada una y comprueba que esté en `_sd`.
- 3. ¿Para qué sirve la sal en cada divulgación?** Para que el resumen de «Ingeniería» no sea siempre el mismo y no se pueda adivinar el contenido de una divulgación oculta probando valores.
- 4. ¿Por qué la credencial lleva `cnf`?** Porque liga la credencial a la **clave pública del titular**. Sin ese enlace, cualquiera con una copia podría presentarla. Con él, hace falta además una firma (KB-JWT) hecha con la clave privada del titular.
- 5. ¿Qué prueba el KB-JWT?** Que quien presenta posee la clave de `cnf` **ahora**, para **este** verificador (`aud`), en **esta** solicitud (`nonce`), y que las divulgaciones presentadas son **exactamente** las cubiertas por `sd_hash`.
- 6. ¿Qué es el código pre-autorizado y para qué el `tx_code`?** El primero va en la oferta y permite pedir el token sin iniciar sesión. Como la oferta puede interceptarse, se añade un `tx_code` corto, entregado por **otro canal**, que debe presentarse junto con el código.
- 7. ¿Para qué la prueba de posesión de emisión (`openid4vci-proof+jwt`)?** Para que el emisor sepa que el solicitante controla la clave que será incluida en `cnf`. Lleva el `c_nonce` (de un solo uso) y la audiencia.
- 8. ¿Por qué la cartera verifica la credencial al recibirla?** Porque no debe guardar lo que no ha comprobado. Resuelve el DID del emisor (campo `kid`) y verifica la firma; así detecta credenciales alteradas o de otro emisor.
- 9. ¿Qué significa «interoperar» y por qué importa el perfil?** Que dos sistemas independientes se entiendan sin ajustes. Para eso no basta con que ambos usen «SD-JWT»: deben coincidir en el `typ`, el algoritmo, la forma de las claves, etc. Por eso el perfil se fija y se documenta.
- 10. ¿Qué es el registro de ejecución y por qué es mejor que un texto?** Una lista de lo que **realmente** se usó, leída de los artefactos producidos. Un texto escrito a mano puede divergir del código sin que nadie lo note.

11. ¿Por qué prohibir protocolos propios? Porque rompen la interoperabilidad: obligan a que los demás implementen **tu** protocolo.

12. ¿Por qué dos claves, una del emisor y otra del titular? Si fueran la misma, el emisor podría fabricar presentaciones en nombre del titular, o cualquiera con una credencial robada podría presentarla. Cada firma debe responder a una pregunta distinta: *¿es auténtica?* y *¿quién la presenta?*

13. ¿Cómo encuentra el verificador la clave del emisor? En el `kid` de la cabecera hay un DID URL; resuelve el DID (ERSo 007), obtiene el DID Document del emisor (ERSo 006) y busca la clave autorizada para `assertionMethod`. EUDI usa certificados `x5c`; este proyecto usa el DID (§27).

14. ¿Qué es DCQL? El lenguaje con el que el verificador dice qué quiere: formato, tipo de credencial y datos (rutas de claims).

24. Preguntas por laboratorio y por criterio

Paso 1 (laboratorio A)

P1. *¿Qué dice el perfil sobre los algoritmos?* → Solo ES256 (JOSE) / COSE -7; todo lo demás se rechaza. **P2.** *¿Qué no está implementado y lo dice el perfil?* → La presentación de mdoc (DeviceResponse).

Criterio 1 (laboratorio B)

P3. *¿Dónde están los datos personales dentro del JWT firmado?* → En ningún lado: solo sus cuatro resúmenes bajo `_sd`. **P4.** *Presentando solo `program`, ¿qué ve el verificador?* → `iss`, `vct` y `program`. Nombre, apellido y promedio no llegan. **P5.** *¿Qué contiene el `kid` de la cabecera?* → El DID URL de la clave del emisor (`...#key-1`). **P6.** *¿Qué muestran `cnf.jwk` y `deviceKey`?* → La misma clave del titular: ambas credenciales quedan ligadas a ella. **P7.** *`tx_code` incorrecto:* → `400 invalid_grant`; con el correcto después, la oferta sigue disponible. **P8.** *Cambiar el `vct` de una credencial:* → «La firma del emisor no es válida». **P9.** *Cambiar una divulgación:* → «Divulgación sin digest en la credencial». **P10.** *Repetir la respuesta de presentación:* → `400`, «La solicitud ya fue respondida». **P11.** *¿Qué no demuestra este criterio?* → Interoperabilidad con terceros.

Criterio 2 (laboratorio B)

P12. *¿Qué contiene una fila del registro de ejecución?* → Rol, formato, `typ`, algoritmo y resumen del artefacto real. **P13.** *¿Qué valor tiene `mismatches` y qué significa?* → `[]`: lo usado coincide con lo declarado. **P14.** *¿Cómo sabemos que el registro detectaría una diferencia?* → La prueba automática lo alimenta con un ES384 y comprueba que aparece.

Criterio 3 (laboratorio D)

P15. *Rutas típicas de protocolo propio:* → todas `404`. **P16.** *Portal de administración por el canal público:* → `404`; por `9443` sin token, `401`; sin certificado, `400`. **P17.** *¿De dónde sale el inventario?* → Del código: cada ruta se registra con `ep(...)`, que la añade al catálogo.

Criterio 4 (laboratorio B)

P18. Firma de la credencial con la clave del titular: → `false`. KB-JWT con la clave del emisor: → `false`. **P19.** ¿Dónde vive cada clave privada? → La del emisor, en el servicio de firma; la del titular, en el hardware seguro del dispositivo. **P20.** ¿Dónde se publica cada clave pública? → La del emisor, en su DID Document; la del titular, dentro de la credencial.

Pruebas automáticas

P21. ¿Cuántas pruebas hay? → 12 (`CredentialsCoreTest`) + 16 (`Erso002CredentialsTest`).

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

25. Redacción modelo de las cuatro respuestas

Criterio 1 — Emisión y recepción en `dc+sd-jwt` y `mso_mdoc`

El emisor OpenID4VCI 1.0 (flujo de código pre-autorizado con `tx_code`) anuncia en sus metadatos ambos formatos con ES256, y emite `dc+sd-jwt` (con `cnf`, `_sd` y divulgaciones) y `mso_mdoc` (MSO firmado en COSE_Sign1). La cartera verifica la firma del emisor resolviendo su DID antes de guardar. Una presentación OpenID4VP (`direct_post`, DCQL) con divulgación selectiva entregó al verificador solo el programa, con KB-JWT (`aud`, `nonce`, `sd_hash`). Se rechazan `tx_code` erróneo, credencial alterada, divulgación ajena y repetición. **Límite:** las pruebas cruzadas son entre componentes propios; no hay prueba contra implementaciones de terceros. **Evidencia:** §17; pruebas `Erso002CredentialsTest > C1 ...`.

Criterio 2 — Declarado = usado

El perfil por formato está declarado en un único lugar (`CredentialProfiles`, `docs/PERFIL-CREDENCIALES.md`): `dc+sd-jwt` y `mso_mdoc`, solo ES256 (COSE –7), resumen sha-256 / SHA-256. El registro de ejecución lee los artefactos efectivamente producidos y verificados y no encontró diferencias (`mismatches: []`); una prueba comprueba que detectaría un algoritmo o formato no declarado. **Evidencia:** §18.

Criterio 3 — Sin protocolo propio paralelo

El inventario de endpoints (11 rutas) declara la norma de cada una; las únicas rutas de credenciales son las de OpenID4VCI y OpenID4VP. Nueve rutas típicas de un protocolo propio devuelven 404; los portales de administración no existen en el canal público y exigen certificado y token por el canal de escritura; nginx admite un solo verbo por ruta. **Evidencia:** §19; prueba `C3 inventario de endpoints ...`.

Criterio 4 — Firma del emisor separada de la prueba del titular

La matriz de claves por rol (generada con las claves reales) muestra dos claves distintas: `ISSUER_SIGNING` (servicio de firma; pública en el DID del emisor) y `HOLDER_PROOF` (hardware del titular; pública en `cnf/deviceKey`). La credencial solo valida con la clave del emisor y el KB-JWT solo con la del titular; `KeyMatrix` falla si comparten clave. **Evidencia:** §20; pruebas `criterio 4 ...`.

26. Lista de comprobación para quien acepta

☐ Qué comprobar	Cómo	Resultado esperado
☐ Perfil documentado	<code>docs/PERFIL-CREDENCIALES.md</code>	Formatos, ES256, «todo lo demás se rechaza»
☐ Metadatos del emisor	recorrido §1	Dos configuraciones, ES256 / –7
☐ Oferta con <code>tx_code</code>	recorrido §2	código pre-autorizado + <code>tx_code</code>
☐ SD-JWT por partes	recorrido §3	Sin datos personales en el JWT; 4 resúmenes en <code>_sd</code> ; <code>kid</code> DID URL; <code>cnf</code>
☐ mdoc verificado	recorrido §4	<code>docType</code> , <code>issuerKid</code> , <code>deviceKey</code> = clave del titular
☐ Credencial alterada	recorrido §5	Rechazo (firma / divulgación)
☐ <code>tx_code</code> erróneo	recorrido §5	<code>400 invalid_grant</code>
☐ Presentación	recorrido §6–7	VERIFIED con solo <code>program</code>
☐ Repetición	recorrido §7	<code>400</code> ya respondida
☐ Registro de ejecución	recorrido §8	<code>mismatches: []</code>
☐ Inventario de endpoints	<code>grep 'ep("'</code>	11 rutas con su norma
☐ Rutas típicas	recorrido §11	404
☐ Portal fuera del canal público	<code>curl</code>	404 / 401 / 400
☐ Matriz de claves	recorrido §9	Dos huellas distintas
☐ Firmas cruzadas	recorrido §10	<code>true, false, true, false</code>
☐ Pruebas automáticas	<code>./gradlew :credentials-core:test</code> <code>:credential-service:test</code>	28/28
☐ Límite entendido	§27	Sin pruebas con terceros

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

27. Lo que este informe NO demuestra, y lo que se encontró

27.1 «Pruebas cruzadas»: alcance real

Emisor, verificador y cartera fueron escritos por el mismo equipo leyendo la misma especificación. Que se entiendan entre sí **no demuestra** interoperabilidad con terceros. El emisor EUDI de referencia (`eudi-srv-pid-issuer`) está en la máquina del equipo, pero **no se ejecutó una prueba contra él**. Es el siguiente paso para que el criterio 1 sea concluyente.

27.2 Diferencias de perfil con los emisores EUDI (analizadas, no todas corregidas)

Tema	EUDI	Este proyecto	Consecuencia
Clave del emisor	<code>x5c</code> (cadena de certificados)	DID del emisor (<code>kid = did:web:...#key-1</code>)	Un verificador EUDI estándar no validaría estas credenciales sin adaptación. Coherente con la arquitectura del equipo, pero no es el perfil HAIP
mdoc: identificación del emisor	<code>x5chain</code> (ISO 18013-5)	<code>kid</code> (COSE etiqueta 4)	Un lector ISO estricto lo rechazaría
typ de SD-JWT VC	<code>dc+sd-jwt</code>	<code>dc+sd-jwt</code> ; <code>vc+sd-jwt</code> se rechaza por defecto	Alineado
Pruebas de posesión	<code>jwt</code> (con key attestation) y <code>attestation</code>	solo <code>jwt</code> , sin key attestation en la prueba	Menos garantía sobre la clave del titular
Cifrado	El emisor «generic» exige JWE	No hay cifrado	Distinto
Tokens	Keycloak + DPoP	Servidor de autorización propio, sin DPoP	Distinto

27.3 Lo que NO se implementó

Presentación de mdoc (DeviceResponse / SessionTranscript de OpenID4VP Anexo B); cifrado JWE; DPoP; endpoints diferido y de notificación; firma de la solicitud OpenID4VP (JAR); `direct_post.jwt` ; revocación (*status list*); persistencia del estado del emisor y del verificador (**en memoria**, se pierde al reiniciar); flujo de código de autorización.

27.4 Otros límites

Límite	Detalle
Detalles normativos de memoria	Forma de la respuesta (<code>credentials: [{credential}]</code>), <code>proofs: {jwt: [...]}</code> , codificación base64url del <code>IssuerSigned</code> , prefijo de cliente <code>redirect_uri:</code> . Confirmar cada uno contra el texto exacto (<code>docs/VERSIONES-NORMATIVAS.md</code>).
Clave del emisor en un archivo	<code>evidencias/work/avance.json</code> (laboratorio). En producción: KMS/HSM («Signature Service»).
Credencial de ejemplo	«Credencial académica» ficticia (<code>vct = urn:avance:credential:academic:1</code> , <code>docType = org.avance.academic.1</code>). No es un esquema aprobado.
Paso 5 de la ERSO	«Corregir diferencias de perfil de las bibliotecas reutilizadas»: no se reutilizó ninguna . Se documentan las diferencias con las bibliotecas EUDI y se tomaron decisiones de perfil.
El token de administración	Lo imprime el simulador solo como parte de los comandos; es de laboratorio.
Fechas	Desarrollo y prueba ya habían pasado cuando se hizo esta implementación.

27.5 Hallazgo heredado: HEAD responde 404 en el registro

Descubierto en la ERSO 005. No afecta a esta ERSO.

28. Si algo no sale como en el informe

Síntoma	Causa probable	Qué hacer
El simulador imprime el uso y no el recorrido	Imagen <code>holder-sim</code> anterior	<code>./gradlew :wallet-sim:installDist</code> y <code>docker compose --profile tools build holder</code>
<code>RESOLUTION_FAILED</code> / la cartera no verifica al emisor	El DID de Avance no está publicado	Ver informe ERSO 006 (publicar entidades/avance)
<code>401</code> en <code>/admin/offers</code>	Falta o es incorrecto <code>CREDENTIAL_ADMIN_TOKEN</code>	Cargar <code>deploy/.env</code>
<code>400</code> de nginx en el portal	Falta el certificado cliente (<code>vdr-admin.p12</code>)	Revisar <code>certs/</code>
<code>invalid_grant</code> al canjear	<code>tx_code</code> erróneo, oferta vencida o ya usada	Crear una oferta nueva
Estado perdido tras reiniciar <code>credential</code>	El estado está en memoria	Repetir el recorrido
Pruebas de servicio no conectan	Falta <code>vdr-test-pg</code>	Levantarla (puerto 55432)

29. Operación diaria

```
cd deploy
docker compose up -d
```

```
# levantar
```

```
docker compose --profile tools down      # bajar (NUNCA con -v)
bash scripts/lab-002-completo.sh         # repetir todos los laboratorios
```

- **Ver el perfil en vivo:** `curl a /.well-known/openid-credential-issuer`.
 - **Ver la evidencia del criterio 2:** `GET /admin/execution-log` (certificado + token).
-

ANEXOS

Anexo A — Los scripts

Script	Para qué sirve
scripts/lab-002-completo.sh	Ejecuta los laboratorios en una sola sesión
scripts/e2e.sh	Recorrido de extremo a extremo (121 comprobaciones)
scripts/generar-pdf.sh	Convierte un informe a PDF

Anexo B — Autocomprobación (con respuestas)

1. ¿Dónde van los datos personales en un SD-JWT? → En divulgaciones aparte; en el JWT solo sus resúmenes.
2. ¿Qué firma el KB-JWT y con qué clave? → {iat, aud, nonce, sd_hash} con la clave de `cnf`.
3. ¿Qué algoritmo admite el perfil? → ES256 (COSE –7).
4. ¿Qué es un protocolo paralelo? → Un camino propio, no estándar, de emisión o presentación.
5. ¿Qué pasa con `typ = vc+sd-jwt`? → Se rechaza por defecto.
6. ¿Quién verifica la firma de la credencial y con qué clave? → Cualquiera, con la clave del DID del emisor.
7. ¿Quién verifica el KB-JWT? → El verificador, con la clave de `cnf`.
8. ¿Qué NO está implementado? → Presentación de mdoc, JWE, DPoP, revocación.
9. ¿Qué no demuestra el criterio 1? → Interoperabilidad con terceros.
10. ¿De dónde sale el inventario de endpoints? → Del código (`ep(...)`).

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. *Descompón el criterio 2.* → Verbo: *coinciden*. Evidencia: documento de perfil y registro de ejecución. Caso negativo: un artefacto con ES384 o formato no declarado. Mecanismo: `ExecutionLog.mismatches`.

Ejercicio 2. *Diseña una prueba para «la divulgación selectiva funciona».* → Emitir una credencial con 4 datos; presentar uno; comprobar que el verificador recibe solo ese; intentar añadir una divulgación ajena y esperar rechazo (§17.5–17.7).

Ejercicio 3. *¿Cómo harías la prueba que falta para el criterio 1?* → Ejecutar el emisor EUDI de referencia, pedirle una credencial con la cartera de este proyecto y presentarla a un verificador de terceros; documentar las diferencias (§27.2).

Anexo D — Referencias

- SD-JWT y SD-JWT VC (IETF OAuth); ISO/IEC 18013-5 (mDL); RFC 9052 (COSE).

- OpenID4VCI 1.0, OpenID4VP 1.0; RFC 6749, RFC 8414.
- W3C Verifiable Credentials Data Model 2.0.
- docs/PERFIL-CREDENCIALES.md , docs/ANALISIS-BASES-EUDI.md , docs/VERSIONES-NORMATIVAS.md , docs/MARCO-CONCEPTUAL.md .